Concurrency-Safe Real-Time Course Registration: An Experimental Evaluation

Oct 29, 2026 · @Srikrishna

Abstract

Row-level locking eliminated seat over-allocation in a live course-registration service, at a cost of 3–11 ms per request. We extended a Go/PostgreSQL student–course–project service with a React dashboard fed by Server-Sent Events (SSE). We then ran a pre-planned experiment with 30 seeded, randomised repetitions per cell. A naive check-then-insert path over-allocated a 5-seat course in 120 of 120 trials, selling up to 62 seats. The locked path never did (0/120; 95% upper bound 2.5%; Fisher $p = 3.4 \times 10^{-17}$). A legacy read-check-save project claim told every one of K concurrent students that they had won. An atomic conditional update always produced exactly one winner. Locking raised median latency 1.6–1.7× (Cliff's $\delta \geq 0.77$). SSE delivered changes to up to 100 viewers in a 4 ms median, compared with 161 ms and 1,626 ms for 500 ms and 2,000 ms polling. All raw data, code hashes and analysis scripts are kept for reproduction.

1. Introduction and literature review

When registration opens, many students compete for a small number of seats within seconds. Any service that checks capacity and then writes is exposed to a time-of-check/time-of-use race. This study measures how often that race causes real damage, and what preventing it costs, in a Go/PostgreSQL course-registration service extended with a live web dashboard.

Isolation anomalies. Berenson et al. showed that the ANSI isolation levels miss several anomalies, including the *lost update* (P4): a transaction reads a value, another transaction writes it, and the first then overwrites that write [1]. PostgreSQL's default level, Read Committed, gives each statement a fresh snapshot. Non-repeatable reads, phantoms and serialisation anomalies therefore remain possible [2]. A capacity check made with `SELECT COUNT(*)` followed by an `INSERT` is exactly this pattern.

Remedies. Two common database-level fixes exist. Pessimistic row locks (`SELECT … FOR UPDATE`) make competing writers wait. PostgreSQL's documentation advises taking such locks in one consistent order to avoid deadlocks [3]. The alternative is a single conditional write whose `WHERE` clause re-checks the condition, or a trigger with a `CHECK` constraint. A practitioner design record for an enrolment system rejects application-level checks for this reason and enforces capacity inside the database [6].

Pushing updates to the browser. Server-Sent Events (SSE) stream `text/event-stream` messages over one long-lived HTTP response. The browser reconnects automatically, and on reconnection it reports the last event id it received [4]. WebSockets give a full-duplex channel instead [7]. Van de Vyvere et al. benchmarked HTTP polling against SSE with 25,000 clients. SSE used less server CPU, delivered updates with lower client latency and served more than twice as many clients. Polling only won when clients accepted a fixed maximum delay [5].

Research gap. Published comparisons of live-update transports focus on throughput at scale. Concurrency guidance for enrolment is mostly informal and rarely quantified. We found no study that measures, with repeated trials and uncertainty estimates, (a) how often a naive enrolment path over-allocates as contention grows, (b) the latency price of the locking fix, and (c) end-to-end staleness of SSE versus polling, all on one system. This report addresses that gap at small scale, using an IMRaD-style structure [13].

2. Design of experiments

All research questions, hypotheses, factor levels, sample sizes and tests below were fixed before any data was collected. Each experiment compares the shipped implementation against a baseline that reproduces the earlier, unprotected logic.

RQ	Question	Arms (baseline vs treatment)	Factor and levels	Repetitions	Primary metric
RQ1	Does row locking stop seat over-allocation?	Naive check-then-insert vs locked (`FOR UPDATE`)	Concurrent students N = 10, 25, 50, 100; capacity C = 5	30 per cell, 240 trials	Over-allocation O = max(0, enrolled − C)
RQ2	Does a conditional `UPDATE` stop lost project claims?	Legacy read-check-save vs atomic `UPDATE … WHERE student_id IS NULL`	Concurrent claimants K = 2, 5, 10, 20	30 per cell, 240 trials	Lost updates L = successes − 1
RQ3	What latency does correctness cost?	Same as RQ1	Same as RQ1	Unit = one trial (mean of its requests)	Per-trial mean and p95 latency (ms)
RQ4	How stale is a viewer's screen?	SSE push vs HTTP polling every 500 ms and 2,000 ms	SSE subscribers S = 1, 10, 50, 100	100 events per SSE level, 40 per polling level	Action-to-screen delay (ms), delivery ratio

Hypotheses. H1: the naive arm over-allocates with a probability that grows with N, and the locked arm never does. H2: the legacy arm reports more than one winner for the same project, and the atomic arm never does. H3: locking raises latency, with an effect that

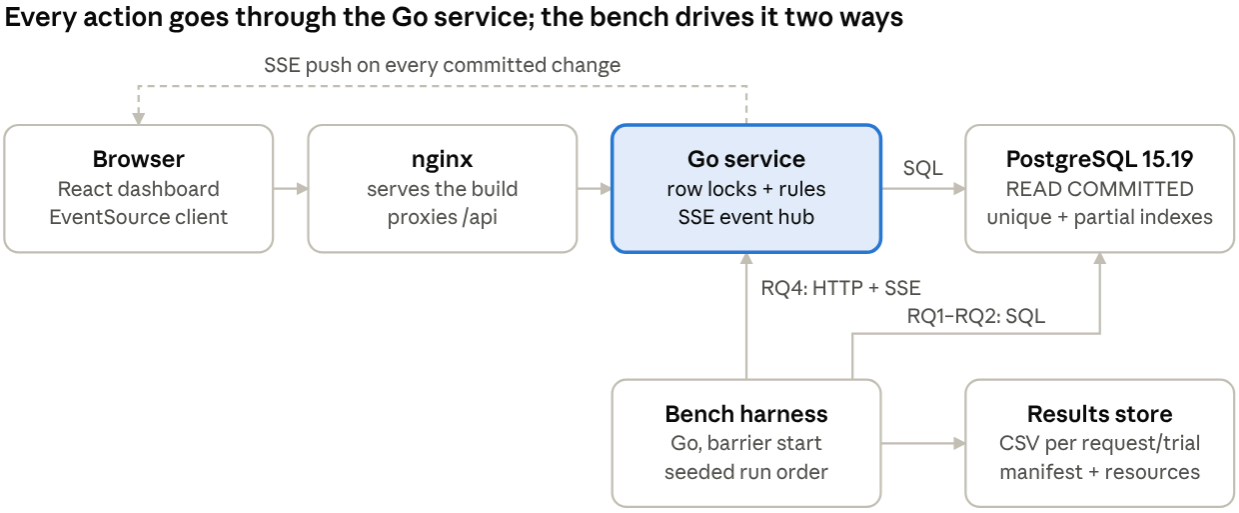
grows with N. H4: the median SSE delay is below that of both polling intervals, and polling delay is roughly Uniform(0, T) plus one round trip.

Controlled conditions. Every trial uses a fresh course or project, so trials are independent. Benchmark students get a 1,000-credit limit and courses carry 1 credit, so only the seat rule can bind. The traffic simulator is off. All goroutines wait at a barrier and are released together. Within each repetition, arms and levels run in random order (seeded), so slow drift cannot favour one arm. Two warm-up trials per cell are discarded. Both arms share one connection pool of 60 and write the same activity row after commit, so they differ only in their locking.

Statistics and sample size. Rates are given with Wilson 95% intervals [11]. Arms are compared with Fisher's exact test (rates) and the Mann–Whitney U test (latency) [12]. p-values are Holm-corrected across levels, with $\alpha = 0.05$. Effect sizes use Cliff's δ, with $|\delta| \geq 0.474$ counted as large [9], and medians carry 10,000-resample bootstrap CIs [10]. Thirty trials per cell were chosen with the rule of three in mind [8]. If an arm records zero failures in 120 trials, the true failure rate is below 2.5% with 95% confidence.

3. System, environment and data management

The system is a layered Go service (chi router, service layer, GORM repositories) in front of PostgreSQL. A React dashboard reaches it through nginx. After each committed change, the service writes an activity row and pushes the event to every open browser over SSE.



System under test and experiment harness · 6 components

The bench harness calls the production service code in-process for RQ1–RQ2. For RQ4 it acts as many browsers against the running container. All experiments ran on one host on 29 October 2026, between 05:00:29 and 05:03:20 UTC.

Component	Version or setting
Host	Intel Core i5-12400F (6 cores, 12 threads), 15.9 GB RAM, Windows 10 Pro 19045
Container runtime	Docker 29.9.2, WSL2 VM with 12 vCPU and 7.7 GiB
Database	PostgreSQL 15.19, default READ COMMITTED isolation
Service and harness	Go 1.20.14, GORM 1.25.5, pgx 5.3.1, chi 5.0.10; DB pool capped at 60
Frontend	React 18.3.1, Vite 5.4.10, nginx 1.27
Analysis	Python 3.14, NumPy 2.5.4, pandas 3.1.0, SciPy 1.18.1, statsmodels 0.15.0

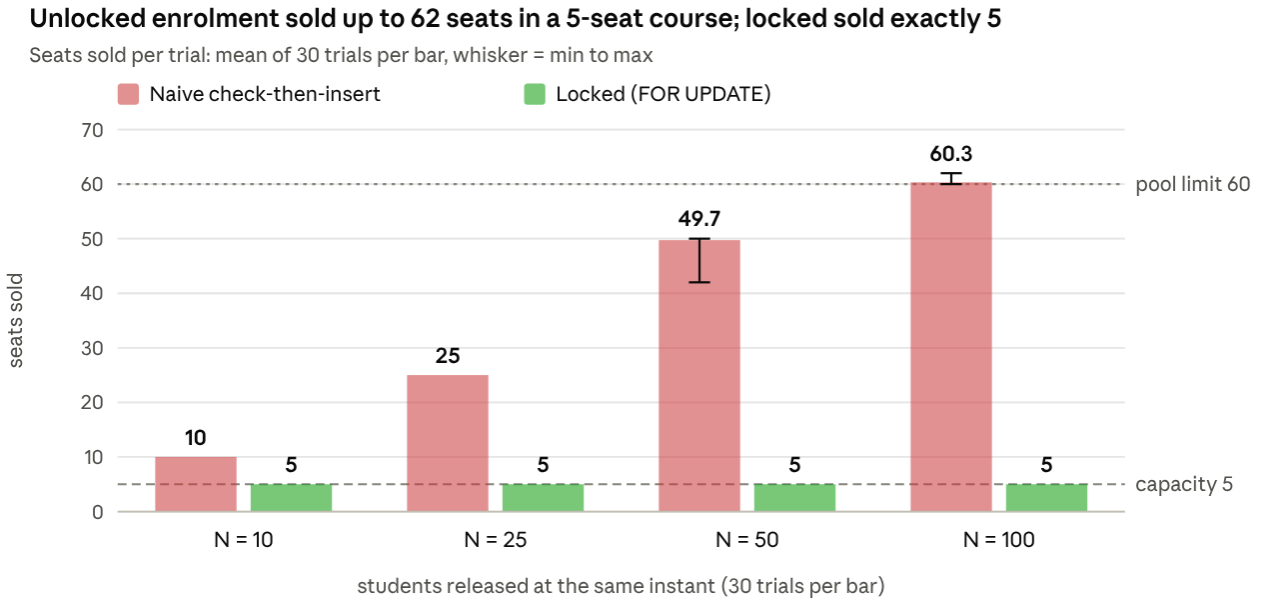
Data collection. Every request is logged with its trial id, arm, level, repetition, warm-up flag, start offset, latency in milliseconds, outcome and any error text: 11,840 rows for RQ1 and 2,368 for RQ2. Every trial is logged with its UTC timestamp, wall time, successes, conflicts, server errors and the final database state (256 trials per experiment). RQ4 adds 16,900 delay records. Container CPU and memory were sampled 53 times during the run.

Validation and storage. The harness reads the final database state, so a reported success can be checked against a real row. All raw CSVs, the `manifest.json` and the derived `summary.json` are kept unedited in `experiments/results/2026-10-29/`. Benchmark rows carry a run tag and are deleted after the run, so the live dashboard's data is untouched.

4. Results

4.1 RQ1: seat over-allocation

All 120 naive trials over-allocated the course. None of the 120 locked trials did. At every N the one-sided Fisher exact test gives $p = 3.4 \times 10^{-17}$ after Holm correction.



experiments/results/2026-10-29/rq1_trials.csv · 240 measured trials, warm-ups excluded

Up to N = 25, the naive arm admitted every student: every transaction counted the seats before any other one committed. Beyond that, over-allocation levelled off near 60, which is the size of the connection pool. In this setup, seats sold ≈ min(N, pool) rather than C. That is a structural limit, not a lucky one. The naive arm completed 4,352 enrolments where only 600 were allowed. Every reported success matched a stored row, so the failure is silent: no error reaches the student or the log. The locked arm sold exactly C seats in 120 of 120 trials. The exact one-sided 95% upper bound on its failure rate is 2.5%.

4.2 RQ2: lost project claims

The legacy path told every concurrent claimant that the project was theirs. The atomic path produced exactly one winner in all 120 trials.

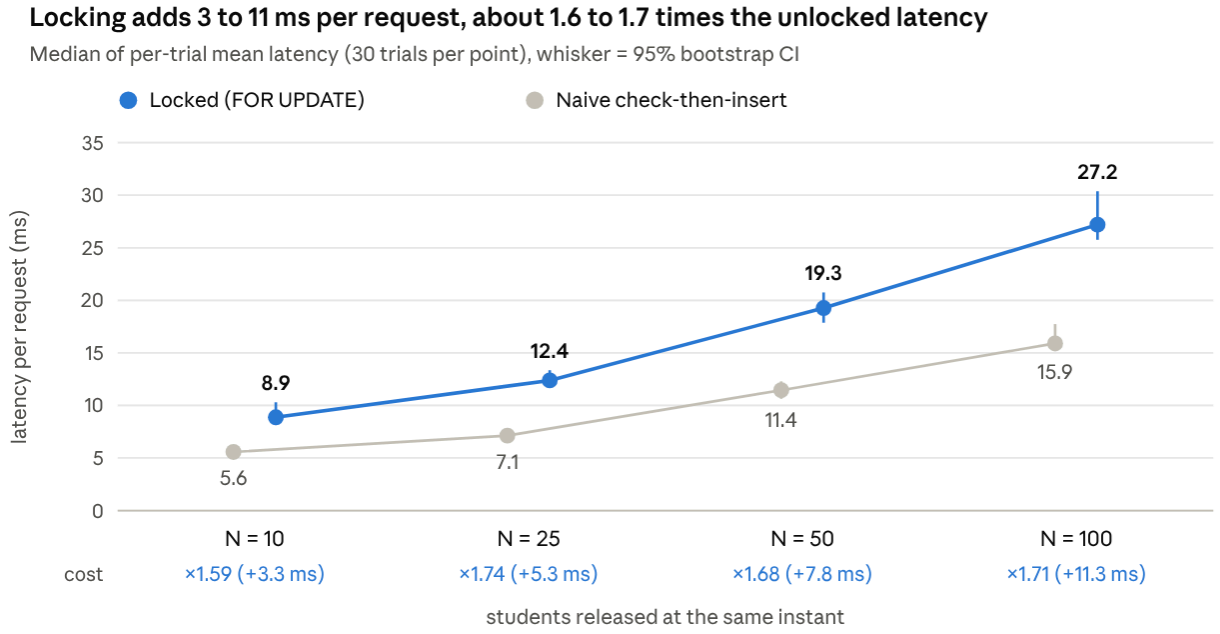
Claimants K	Legacy: "assigned" replies per trial	Legacy: trials with a lost update (95% CI)	Atomic: "assigned" replies per trial	Atomic: trials with a lost update	Holm-adjusted p
2	2.0	30 / 30 (88.6–100%)	1.0	0 / 30	3.4×10^{-17}
5	5.0	30 / 30 (88.6–100%)	1.0	0 / 30	3.4×10^{-17}
10	10.0	30 / 30 (88.6–100%)	1.0	0 / 30	3.4×10^{-17}
20	20.0	30 / 30 (88.6–100%)	1.0	0 / 30	3.4×10^{-17}

In the legacy arm, 990 students across all trials were told they owned a project that the database gave to someone else. Their claims were overwritten, which is Berenson's lost update [1]. No server error occurred, because the partial unique index stops one student

holding two projects but not two students claiming one. The original API would therefore have answered 200 OK to every one of them. The conditional `UPDATE … WHERE student_id IS NULL` converts the race into one success and K − 1 clean 409 conflicts.

4.3 RQ3: the latency cost of correctness

Locking made enrolment 1.59–1.74× slower at the median, at every N. The absolute penalty grew from 3.3 ms at N = 10 to 11.3 ms at N = 100.



experiments/results/2026-10-29/rq1_requests.csv · 9,300 measured requests grouped into 240 trials

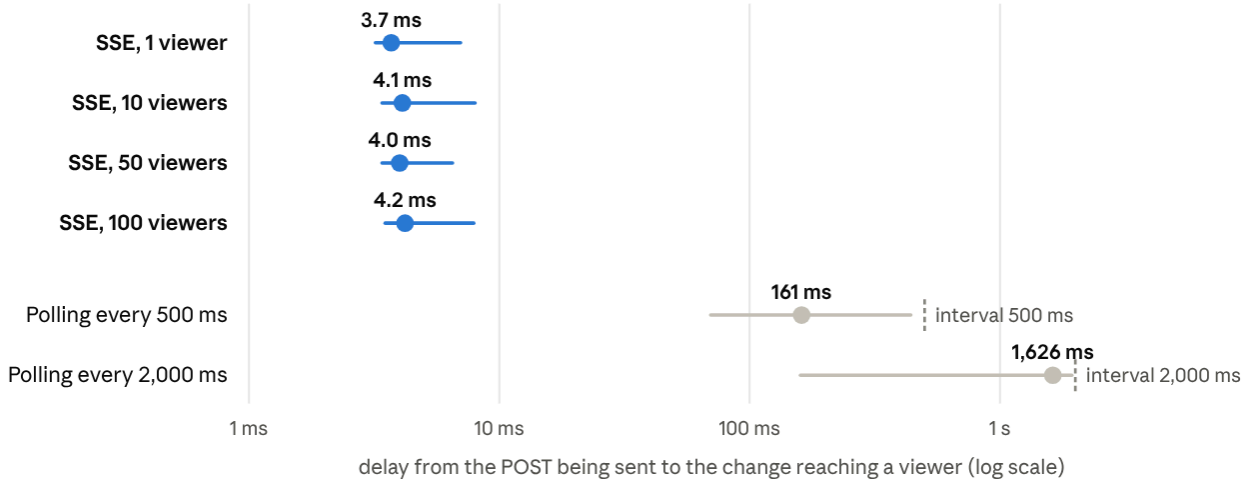
At every level, the difference is large (Cliff's δ = 0.77, 0.80, 0.91 and 0.96) and significant (Mann–Whitney, Holm-adjusted $p \leqslant 3.0 \times 10^{-7}$). The tail grows faster than the median: the per-trial p95 rose from 10.9 to 39.6 ms with locks, against 5.8 to 18.9 ms without. The bootstrap CIs of the latency ratio overlap at all four levels (1.35–1.88 at N = 10, 1.52–1.93 at N = 100). H3 is therefore supported for the absolute cost but not for the relative one. The naive arm stayed faster even though it wrote 7.3× more enrolment rows, so the gap is caused by waiting on the locks, not by extra work.

4.4 RQ4: how quickly other viewers see a change

With SSE, a change reached other screens in 3.7–4.2 ms at the median, with a p99 of 8.0 ms or less at up to 100 viewers. All 16,900 updates were delivered.

SSE reached every viewer in about 4 ms; polling took 40 to 400 times longer

Delay from action to screen, log scale. Dot = median, whisker = 5th to 99th percentile. All 16,900 updates arrived.



experiments/results/2026-10-29/rq4_delays.csv · 16,100 SSE and 800 polling deliveries

The acting user's own POST took about 4 ms, so other viewers saw a change at about the moment its author did. Going from 1 to 100 subscribers raised the SSE median by 0.5 ms. That trend is statistically detectable but practically negligible (Spearman $\rho = 0.15$). Polling every 500 ms gave a 161 ms median; polling every 2,000 ms gave 1,626 ms, which is 39× and 396× the SSE median. The distributions do not overlap (Cliff's $\delta = -1.0$, Mann–Whitney $p < 10^{-180}$). H4's first part is supported. Its second part, the Uniform(0, T) model of polling delay, is rejected (Kolmogorov–Smirnov $D = 0.29$ and 0.36, $p < 10^{-29}$). Section 5 explains why.

5. Statistical validity, uncertainty and repeated testing

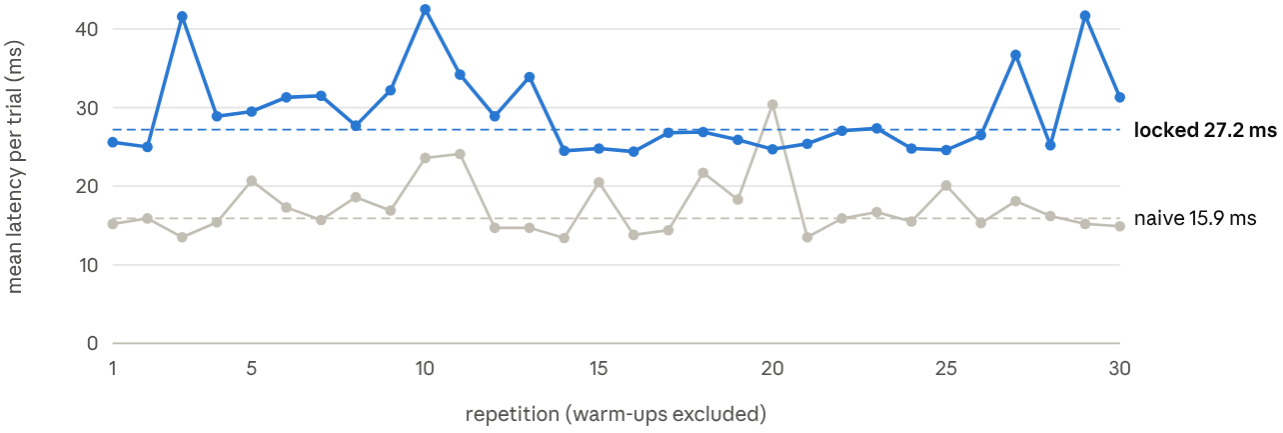
The correctness results are far beyond chance. The latency results hold up across repetitions. One polling estimate is less certain than its sample size suggests.

Unit of analysis. Requests inside one trial compete for the same rows, so they are not independent. All latency tests therefore treat one trial as one observation (n = 30 per cell, 240 per experiment), not the 9,300 individual requests. Treating each request as an independent sample would be pseudo-replication and would overstate the evidence.

How likely is chance? If both arms over-allocated equally often, getting 30 of 30 against 0 of 30 would have probability 3.4×10^{-17} per level. For the zero-failure arms, the exact 95% upper bound is 2.5% (locked enrolment, 0/120) and 2.5% (atomic assignment, 0/120). These are the defensible claims: "zero observed" is not "impossible", only "rarer than 1 in 40 with 95% confidence" at these levels.

Across 30 repetitions at N = 100, the locked arm was slower in 29

Mean request latency per trial, in run order. Locked slower in 29 of 30 repetitions; dashed lines = medians.



experiments/results/2026-10-29/rq1_requests.csv · N = 100 cells, 60 trials

Iteration stability. Arms and levels ran in a seeded random order across 32 repetitions, of which the first 2 were discarded as warm-ups (16 trials per experiment). The locked arm was slower in 28, 28, 29 and 29 of 30 paired repetitions at N = 10, 25, 50 and 100. Over-allocation was identical in all 30 repetitions for N ⩽ 25 and stayed within 60–62 seats at N = 100. Run-to-run spread was moderate: the coefficient of variation of per-trial latency at N = 100 was 0.18 (locked) and 0.22 (naive). At most 2 trials per cell exceeded 3× their cell median (worst case 456 ms, locked, N = 50). That heavy tail is why medians, bootstrap intervals and rank tests were used instead of means and t-tests.

A caveat on polling. Each poller kept one fixed phase for a whole level, and the next action was sent 50–150 ms after the slowest poller had seen the previous one. The action times were therefore locked to the polling clock. Delay varied little within a poller (median SD 29–31 ms) but a lot between pollers (SD of their medians 100 ms at T = 500 and 748 ms at T = 2,000). In effect there are 10 independent phases per interval, not 400 samples. This explains the failed uniform fit and means the polling CIs are too narrow. The SSE-versus-polling conclusion is unaffected: the fastest polling delay (p5 = 70 ms) was still 9× the slowest SSE p99 (8.0 ms).

Resource use. Across 53 samples during the run, the Go service never exceeded 4.7% CPU or 16.8 MiB of memory. PostgreSQL peaked at 86% CPU during the N = 100 bursts, with a median of 1.1%.

6. Real-time visualisation and reproducibility

The system ships its own live dashboard. Every number in this report can be regenerated from stored raw data with two commands.

Live visualisation. The React dashboard shows seat fill per course, waitlists, open projects, connected viewers and an activity feed. It refreshes from SSE events within milliseconds, as measured in RQ4. A built-in traffic simulator drives realistic enrol, drop, waitlist and project actions through the same business rules, so the dashboard can be watched under load. During the experiments, progress was logged per repetition and container load was sampled about every 3 s. The charts in this report are drawn from the stored rows, not from a live feed of the experiment.

Evidence kept for independent verification.

Artefact	Where	Purpose
Raw CSVs (31,830 rows)	`experiments/results/2026-10-29/`	every request, trial, delivery and resource sample, unedited
`manifest.json`	same folder	Go 1.20.14, PostgreSQL 15.19, 12 CPUs, all parameters, seed 20261029
Code fingerprint	`manifest.json` → `source_sha256`	SHA-256 over all 12 `.go` files (`ad717c4fa68a…`), re-verified against the code after analysis
Harness	`experiments/bench/main.go`	baselines, barrier start, seeded run order, cleanup
Analysis	`experiments/analysis/analyze.py` + pinned `requirements.txt`	every statistic quoted here, written to `summary.json`
Rerun guide	`experiments/README.md`	commands, flags and output schema

The project folder is not yet under version control. Running `git init` and tagging this commit would add a second, standard code identity next to the hash.

7. Findings, limitations and impact

Database-level concurrency control is necessary, not optional. It costs about 11 ms at 100 simultaneous requests, and SSE makes the results visible to every viewer in about 4 ms.

Hypothesis	Verdict	Key evidence
H1: naive over-allocates, more so as N grows; locked never does	Supported	120/120 vs 0/120 trials; Spearman ρ = 0.99 between N and seats over-sold
H2: legacy reports several winners; atomic exactly one	Supported	120/120 vs 0/120 trials; 990 students wrongly told they owned a project
H3: locking costs latency, growing with N	Partly supported	absolute cost rose from 3.3 to 11.3 ms; relative cost stayed at 1.6–1.7×
H4: SSE beats polling; polling delay ≈ Uniform(0, T)	Partly supported	SSE median 39× and 396× faster; uniform model rejected because of a phase-locking flaw in the design

Limitations. All runs used one machine over a local network, so real internet latency is missing. The 60-connection pool shaped the naive plateau, and a different pool size would move it. The workload was synthetic: identical requests, one-credit courses and one run date. Only READ COMMITTED was tested. SSE was measured up to 100 viewers, not the thousands of a real campus. In RQ4 the polling actions were phase-locked, which reduced the effective polling sample to 10 phases per interval. Benchmark students accumulated enrolments over the run, which slowly lengthened the credit-sum query; randomised run order spread that drift evenly over both arms.

Practical impact. A registration rush against an unprotected check-then-insert would silently sell seats it does not have. It would also tell students they hold projects they do not, and no error would be raised. The fix adds milliseconds, well below typical web round-trip times. SSE kept the service under 5% CPU while giving viewers sub-10 ms updates, which makes it a sound default for read-mostly dashboards [5].

Future work. (1) Compare row locks with SERIALIZABLE-plus-retry and a trigger-maintained counter with a `CHECK` constraint [6]. (2) Scale SSE to thousands of subscribers and several service instances, using PostgreSQL `LISTEN/NOTIFY` for fan-out. (3) Redesign RQ4 with Poisson-timed actions so polling samples are independent. (4) Repeat the runs on different hardware with emulated network delay. (5) Stream experiment progress into the dashboard itself.

References

1. H. Berenson, P. Bernstein, J. Gray, J. Melton, E. O'Neil, P. O'Neil. A Critique of ANSI SQL Isolation Levels. Proc. ACM SIGMOD, pp. 1–10, 1995.

2. PostgreSQL Global Development Group. Transaction Isolation. PostgreSQL 15 Documentation, §13.2.
3. PostgreSQL Global Development Group. Explicit Locking. PostgreSQL 15 Documentation, §13.3.
4. WHATWG. Server-sent events. HTML Living Standard, §9.2.
5. B. Van de Vyvere, P. Colpaert, R. Verborgh. Comparing a Polling and Push-Based Approach for Live Open Data Interfaces. Proc. ICWE 2020, LNCS 12128, Springer, pp. 87–101.
6. Curriculab. Optimistic concurrency for enrollment (architecture decision record). Practitioner documentation.
7. I. Fette, A. Melnikov. RFC 6455: The WebSocket Protocol. IETF, December 2011.
8. J. A. Hanley, A. Lippman-Hand. If nothing goes wrong, is everything all right? JAMA 249(13):1743–1745, 1983.
9. J. Romano, J. D. Kromrey, J. Coraggio, J. Skowronek. Appropriate statistics for ordinal level data. Annual meeting of the Florida Association of Institutional Research, 2006.
10. B. Efron, R. J. Tibshirani. An Introduction to the Bootstrap. Chapman & Hall, 1993.
11. E. B. Wilson. Probable Inference, the Law of Succession, and Statistical Inference. JASA 22(158):209–212, 1927.
12. H. B. Mann, D. R. Whitney. On a Test of Whether One of Two Random Variables is Stochastically Larger than the Other. Annals of Mathematical Statistics 18(1):50–60, 1947.
13. George Mason University Writing Center. Scientific (IMRaD) Research Reports.